

SQL to Aggregation Mapping Chart

The aggregation pipeline allows MongoDB to provide native aggregation capabilities that corresponds to many common data aggregation operations in SQL.

The following table provides an overview of common SQL aggregation terms, functions, and concepts and the corresponding MongoDB aggregation operators:

SQL Terms, Functions, and Concepts	MongoDB Aggregation Operators
WHERE	<code>\$match</code>
GROUP BY	<code>\$group</code>
HAVING	<code>\$match</code>
SELECT	<code>\$project</code>
ORDER BY	<code>\$sort</code>
LIMIT	<code>\$limit</code>
SUM()	<code>\$sum</code>
COUNT()	<code>\$sum</code> <code>\$sortByCount</code>
join	<code>\$lookup</code>
SELECT INTO NEW_TABLE	<code>\$out</code>
MERGE INTO TABLE	<code>\$merge</code>
UNION ALL	<code>\$unionWith</code>

For a list of all aggregation pipeline and expression operators, see:

- [Aggregation Stages](#)
- [Aggregation Operators](#)

TIP

See also:

[SQL to MongoDB Mapping Chart](#)

Examples

The following table presents a quick reference of SQL aggregation statements and the corresponding MongoDB statements. The examples in the table assume the following conditions:

- The SQL examples assume *two* tables, `orders` and `order_lineitem` that join by the `order_lineitem.order_id` and the `orders.id` columns.
- The MongoDB examples assume *one* collection `orders` that contain documents of the following prototype:

```
{
  cust_id: "abc123",
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),
  status: 'A',
  price: 50,
  items: [ { sku: "xxx", qty: 25, price: 1 },
            { sku: "yyy", qty: 25, price: 1 } ]
}
```

SQL Example	MongoDB Example	Description
<pre>SELECT COUNT(*) AS count FROM orders</pre>	<pre>db.orders.aggregate([{ \$group: { _id: null, count: { \$sum: 1 } } }])</pre>	Count all records from <code>orders</code>
<pre>SELECT SUM(price) AS total FROM orders</pre>	<pre>db.orders.aggregate([{ \$group: { _id: null, total: { \$sum: "\$price" } } }])</pre>	Sum the <code>price</code> field from <code>orders</code>
<pre>SELECT cust_id, SUM(price) AS total FROM orders GROUP BY cust_id</pre>	<pre>db.orders.aggregate([{ \$group: { _id: "\$cust_id", total: { \$sum: "\$price" } } }])</pre>	For each unique <code>cust_id</code> sum the <code>price</code> field.
<pre>SELECT cust_id, SUM(price) AS total FROM orders GROUP BY cust_id ORDER BY total</pre>	<pre>db.orders.aggregate([{ \$group: { _id: "\$cust_id", total: { \$sum: "\$price" } } },</pre>	For each unique <code>cust_id</code> sum the <code>price</code> field, result

SQL Example	MongoDB Example	Description
	<pre>{ \$sort: { total: 1 } }]</pre>	sorted by sum.
<pre>SELECT cust_id, ord_date, SUM(price) AS total FROM orders GROUP BY cust_id, ord_date</pre>	<pre>db.orders.aggregate([{ \$group: { _id: { cust_id: "\$cust_id", ord_date: { \$dateToString: { format: "%Y-%m-%d", date: "\$ord_date" }} }, total: { \$sum: "\$price" } } }])</pre>	For each unique <code>cust_id</code> and <code>ord_date</code> grouping, sum the <code>price</code> field. Excludes the time portion of the date.
<pre>SELECT cust_id, count(*) FROM orders GROUP BY cust_id HAVING count(*) > 1</pre>	<pre>db.orders.aggregate([{ \$group: { _id: "\$cust_id", count: { \$sum: 1 } } }, { \$match: { count: { \$gt: 1 } } }]</pre>	For <code>cust_id</code> with multiple records, return the <code>cust_id</code> and the corresponding record count.
<pre>SELECT cust_id, ord_date, SUM(price) AS total FROM orders GROUP BY cust_id, ord_date HAVING total > 250</pre>	<pre>db.orders.aggregate([{ \$group: { _id: { cust_id: "\$cust_id", ord_date: { \$dateToString: { format: "%Y-%m-%d", date: "\$ord_date" }} }, total: { \$sum: "\$price" } } }, { \$match: { total: { \$gt: 250 } } }]</pre>	For each unique <code>cust_id</code> and <code>ord_date</code> grouping, sum the <code>price</code> field and return only where the sum is greater than 250. Excludes the time portion of the date.
<pre>SELECT cust_id, SUM(price) as total FROM orders WHERE status = 'A'</pre>	<pre>db.orders.aggregate([{ \$match: { status: 'A' } }, { \$group: {</pre>	For each unique <code>cust_id</code> with

SQL Example	MongoDB Example	Description
<pre>GROUP BY cust_id</pre>	<pre>_id: "\$cust_id", total: { \$sum: "\$price" } } }])</pre>	status A, sum the price field.
<pre>SELECT cust_id, SUM(price) as total FROM orders WHERE status = 'A' GROUP BY cust_id HAVING total > 250</pre>	<pre>db.orders.aggregate([{ \$match: { status: 'A' } }, { \$group: { _id: "\$cust_id", total: { \$sum: "\$price" } } }, { \$match: { total: { \$gt: 250 } } }])</pre>	For each unique cust_id with status A, sum the price field and return only where the sum is greater than 250.
<pre>SELECT cust_id, SUM(li.qty) as qty FROM orders o, order_lineitem li WHERE li.order_id = o.id GROUP BY cust_id</pre>	<pre>db.orders.aggregate([{ \$unwind: "\$items" }, { \$group: { _id: "\$cust_id", qty: { \$sum: "\$items.qty" } } }])</pre>	For each unique cust_id sum the corresponding line item qty fields associated with the orders.
<pre>SELECT COUNT(*) FROM (SELECT cust_id, ord_date FROM orders GROUP BY cust_id, ord_date) as DerivedTable</pre>	<pre>db.orders.aggregate([{ \$group: { _id: { cust_id: "\$cust_id", ord_date: { \$dateToString: { format: "%Y-%m-%d", date: "\$ord_date" } } } } }, { \$group: { _id: null, count: { \$sum: 1 } } }])</pre>	Count the number of distinct cust_id ord_date groupings. Excludes the time portion of the date.

TIP

See also:

- [SQL to MongoDB Mapping Chart](#)
- `db.collection.aggregate()`
- [Aggregation Stages](#)

